

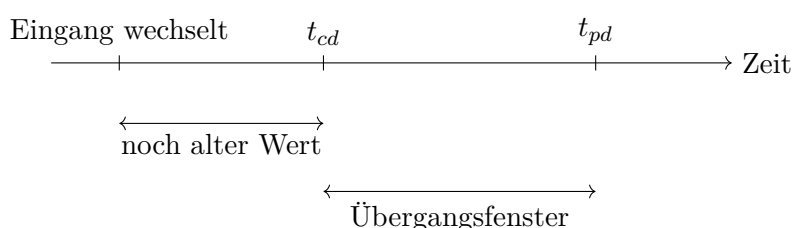
01 / LOGIK BRAUCHT ZEIT

Tag 10: Warum ein richtiges Ergebnis kurz falsch sein kann

Die Wahrheitstabelle sagt dir, welcher Ausgang sich bei festen Eingängen einstellt. Sie sagt nicht, wie sich der Ausgang während eines Eingangswechsels verhält. Reale Gatter reagieren verzögert: Spannungen müssen sich ändern und Kapazitäten werden geladen oder entladen.

Zwei Angaben, zwei verschiedene Fragen

- **Contamination Delay t_{cd} :** Wie früh kann sich der Ausgang nach einem Eingangswechsel erstmals ändern? Es ist die garantierte Mindestverzögerung.
- **Propagation Delay t_{pd} :** Nach welcher Zeit ist der Ausgang spätestens beim neuen gültigen Wert angekommen? Es ist die garantierte Maximalverzögerung unter den angegebenen Bedingungen.



Die Zeitangaben werden vom Eingangswechsel aus gerechnet. Im Übergangsfenster kann der Ausgang bereits wechseln oder kurz mehrfach umschalten. Nach t_{pd} gilt der neue Wert, sofern die Eingänge stabil bleiben.

Beispiel

Ein Gatter hat $t_{cd} = 1\text{ ns}$ und $t_{pd} = 4\text{ ns}$. Sein Eingang wechselt bei 10 ns. Vor 11 ns bleibt der Ausgang beim alten Wert. Zwischen 11 und 14 ns kann er sich ändern. Ab 14 ns ist der neue Wert garantiert.

Die beiden Zahlen bedeuten nicht: „jedes Gatter braucht genau 1 oder genau 4 ns“. Sie begrenzen das mögliche Zeitverhalten. Eine einzelne Messung liefert nicht automatisch garantierte Minima und Maxima.

So liest du eine gemessene Flanke

Eine gebräuchliche Verzögerungsmessung vergleicht die 50%-Punkte von Eingangs- und Ausgangsflanke. Die Flankenzeit beschreibt dagegen das Steigen oder Fallen *eines* Signals, oft zwischen 10% und 90%. Verzögerung und Flankenzeit sind verschiedene Größen.

02 / PFADE DURCH DIE SCHALTUNG

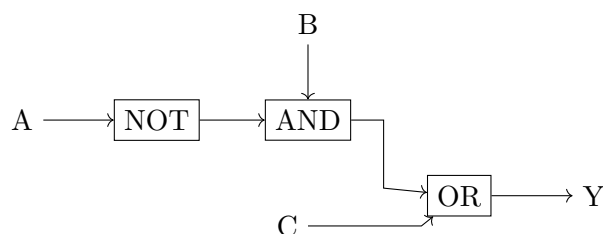
Verzögerungen addieren

Ein Signal durchläuft mehrere Gatter. Für jeden Eingang-Ausgang-Pfad addierst du die Verzögerungen der durchlaufenen Gatter. In diesen Übungen vernachlässigen wir Leitungsverzögerungen.

Gatter	t_{cd}	t_{pd}
NOT	1 ns	3 ns
AND	2 ns	5 ns
OR	1 ns	4 ns

Diese Zahlen sind **eigene Übungswerte**, keine Daten eines bestimmten Bauteils.

Ein direkter und ein längerer Weg



Hier lautet die Funktion $Y = \overline{A}B + C$. Die Pfade sind:

Pfad	Summe t_{cd}	Summe t_{pd}
A: NOT, AND, OR	$1 + 2 + 1 = 4 \text{ ns}$	$3 + 5 + 4 = 12 \text{ ns}$
B: AND, OR	$2 + 1 = 3 \text{ ns}$	$5 + 4 = 9 \text{ ns}$
C: OR	1 ns	4 ns

$$t_{pd, \text{Schaltung}} = \max(12, 9, 4) = 12 \text{ ns}$$

$$t_{cd, \text{Schaltung}} = \min(4, 3, 1) = 1 \text{ ns}$$

Was bedeutet kritischer Pfad?

Der Pfad mit der größten Summe der maximalen Verzögerungen begrenzt, wann du das Ergebnis sicher verwenden kannst. Hier ist es der Weg von A. Die kleinste Summe der Mindestverzögerungen bestimmt, wie früh sich der Ausgang ändern könnte.

Nicht alle Gatter der gesamten Schaltung addieren. Erst die einzelnen Wege verfolgen. Und: Ein Weg wirkt nur auf Y, wenn andere Eingänge ihn nicht blockieren; zum Beispiel blockiert $C=1$ hier Änderungen von A.

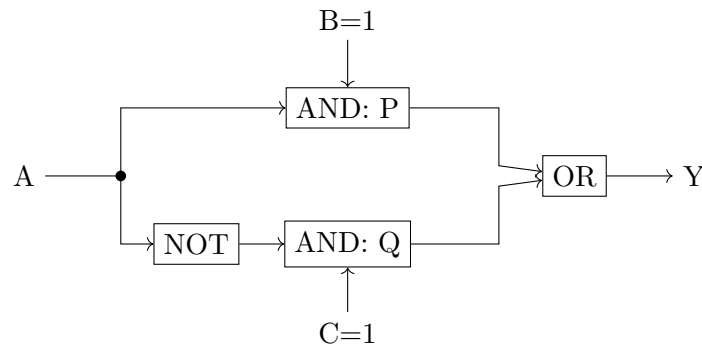
03 / DER STATISCHE 1-HAZARD

Eine Lücke zwischen zwei Wegen

Wir betrachten durchgehend dieselbe Beispielschaltung:

$$P = AB, \quad Q = \overline{A}C, \quad Y = P + Q$$

B und C bleiben 1. A fällt von 1 auf 0. Vorher ist $P=1$, $Q=0$; nachher ist $P=0$, $Q=1$. In beiden stabilen Zuständen soll $Y=1$ sein.



Der zeitliche Ablauf im einfachen Python-Modell

Jetzt verwenden wir **feste** Verzögerungen: NOT 3 ns, AND 2 ns, OR 1 ns. Sie sind keine t_{cd}/t_{pd} -Grenzen. A fällt bei 20 ns.

Zeit	Ereignis
20 ns	A fällt auf 0.
22 ns	P fällt auf 0: direkter AND-Weg, 2 ns später.
23 ns	NOT A steigt auf 1; Y fällt nach dem OR auf 0.
25 ns	Q steigt auf 1: NOT plus AND, insgesamt 5 ns.
26 ns	Y steigt nach dem OR wieder auf 1.

Der Ausgang ist von 23 bis 26 ns LOW. Der kurze unerwünschte Impuls heißt **Glitch**. Die Schaltung hat hier einen **statischen 1-Hazard**: Sie kann kurz auf 0 fallen, obwohl ihr stabiler Ausgang 1 bleiben soll.

Die Wahrheitstabelle ist trotzdem richtig. Das Problem entsteht während des Übergangs, weil P schon ausschaltet, bevor Q einschaltet.

04 / EINE ÜBERBRÜCKUNG HINZUFÜGEN

Gleiche Funktion, sicherer Übergang

$$Y_{\text{mit Zusatz}} = AB + \overline{A}C + BC$$

Der zusätzliche Term $R = BC$ bleibt während des untersuchten Wechsels 1, denn B und C bleiben 1. Auch wenn P und Q beide kurz 0 sind, hält R den OR-Ausgang HIGH.

Warum ändert BC die Wahrheitstabelle nicht?

Wenn $BC=0$, trägt der Zusatz nichts bei. Wenn $BC=1$, gilt $B=C=1$. Dann ist entweder $A=1$ und $AB=1$ oder $A=0$ und $\overline{A}C = 1$. Der Ausgang war also ohnehin 1. BC fügt keine neue Einerzeile hinzu.

$$AB + \overline{A}C + BC = AB + \overline{A}C$$

Das ist das Konsenstheorem. „Redundant“ bedeutet hier: für die **stabile Logikfunktion** überflüssig. Für das Zeitverhalten kann der zusätzliche Weg nützlich sein.

Das bekannte Bild in der Karnaugh-Karte

A / BC	00	01	11	10
0	0	1	1	0
1	0	0	1	1

Die obere waagerechte Gruppe ist $\overline{A}C$, die untere ist AB . Beim Wechsel A:1 nach 0 mit $BC=11$ wechselst du von Index 7 zu Index 3. Diese benachbarten Zellen liegen zunächst in getrennten Gruppen. Die gestrichelte senkrechte Gruppe BC deckt beide gemeinsam ab.

Was du umbauen musst

Ergänze ein AND für B und C und führe dessen Ausgang an einen dritten Eingang des abschließenden OR. Die bisherigen Wege bleiben bestehen. B und C werden von ihren vorhandenen Eingabepins verzweigt.

Diese Maßnahme beseitigt den hier betrachteten statischen 1-Hazard bei einem einzelnen Eingangswechsel. Sie ist keine allgemeine Garantie bei gleichzeitig wechselnden Eingängen, beliebigen Schaltungsumbauten oder allen Arten von Glitches.

05 / PYTHON MACHT DIE ZEIT SICHTBAR

Eine Liste ist ein Signalverlauf

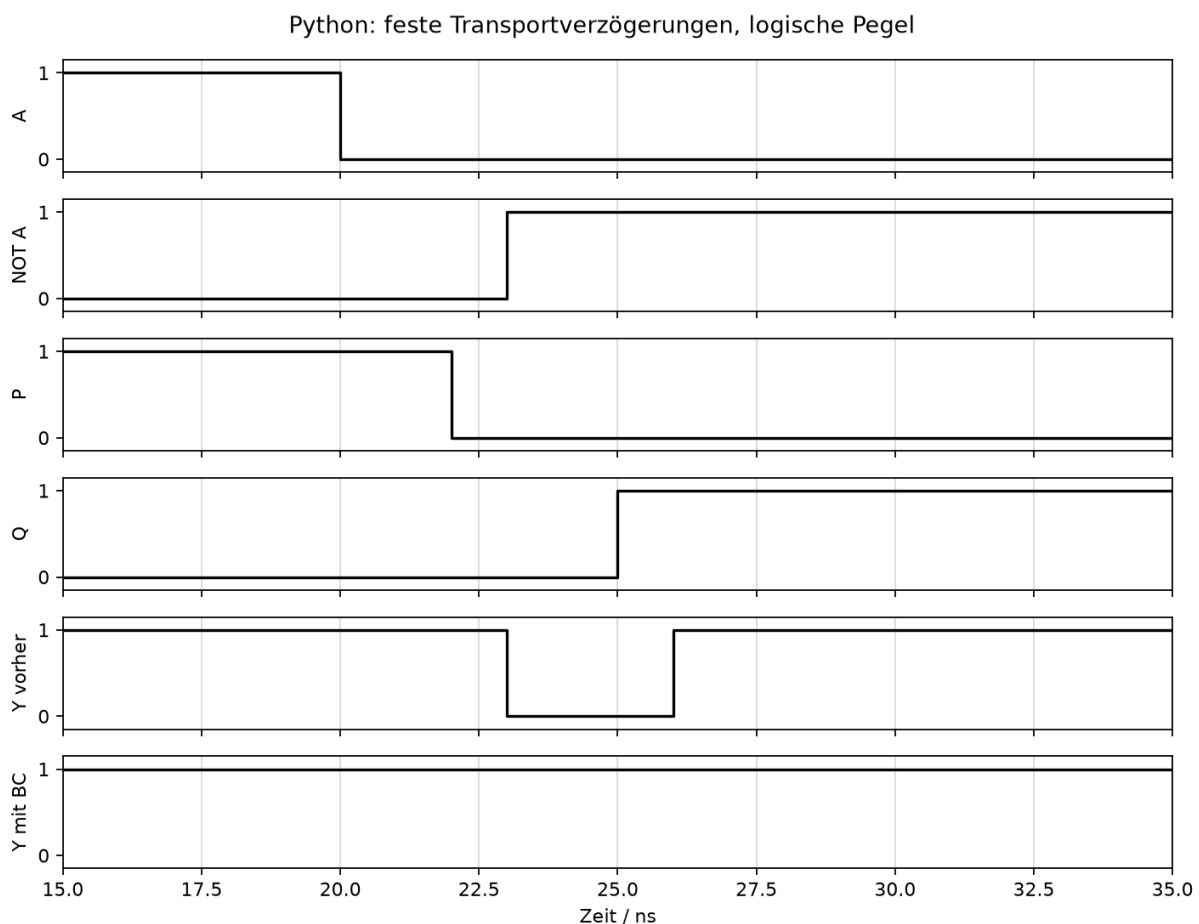
In `timing_einfach.py` steht ein Listeneintrag für eine Nanosekunde. Der Zugriff `a[22]` bedeutet: Wert von A bei 22 ns. Die Funktion `frueher` liest einen Wert aus der Vergangenheit.

```
index = zeit - verzoeigerung
return signal[index]
```

Bei `zeit = 25` und `verzoeigerung = 3` liest du Index 22. Ein verzögertes Gatter reagiert also jetzt auf das frühere Eingangssignal. Für negative Indizes nutzt die Datei ausdrücklich den stabilen Anfangswert, nicht Python's Rückwärtszählung vom Listenende.

Die Lernreihenfolge im Code

1. Lies nur und, oder und nicht. Sage ihre Ausgaben voraus.
2. Lies `frueher`: Warum steht dort eine Subtraktion?
3. Verfolge A, NOT A, P und Q für die Zeiten 20 bis 26 ns.
4. Erst danach lies die Berechnung von Y und des korrigierten Ausgangs.



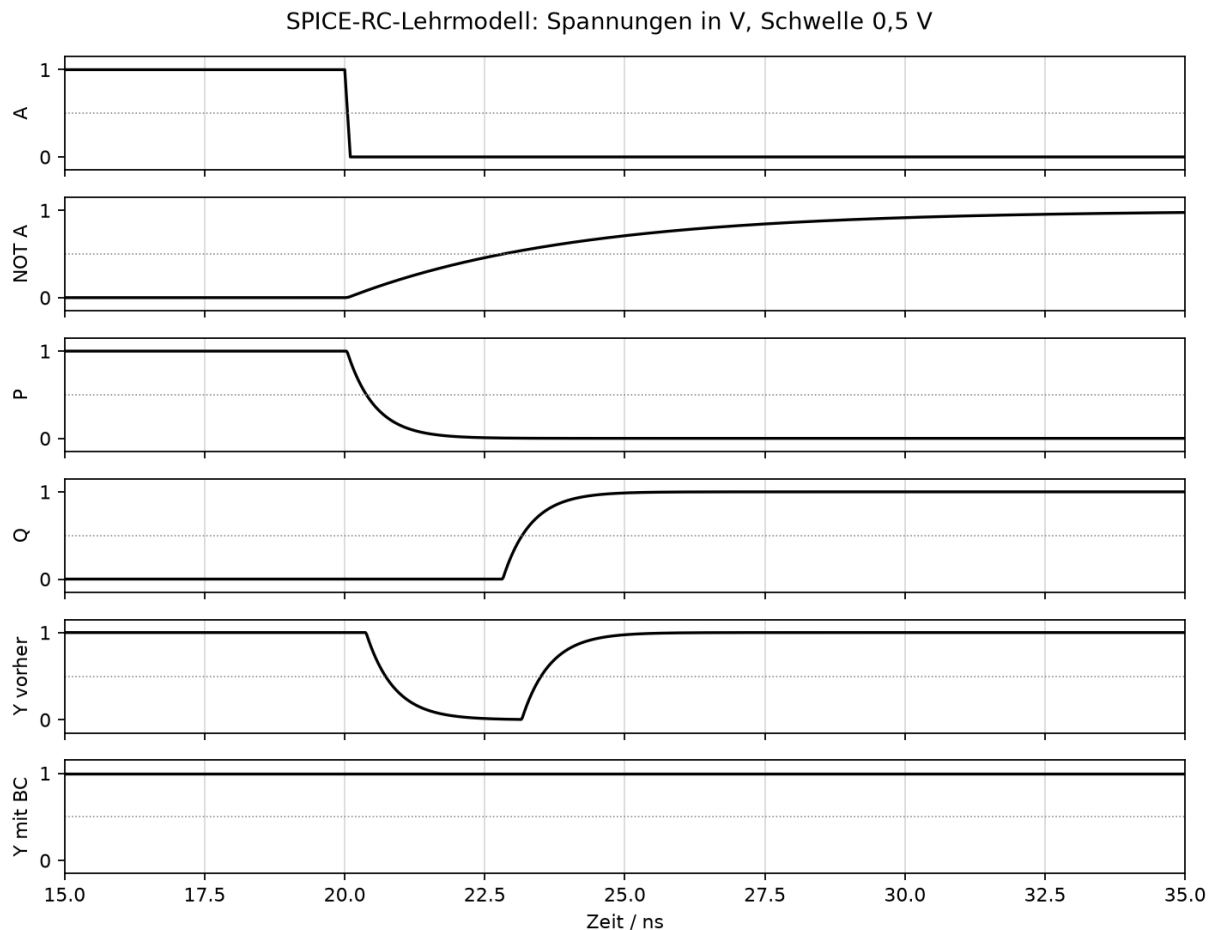
Grenze des Modells: Die Verzögerung wird nur verschoben; kurze Impulse werden nicht ausgefiltert. Das heißt Transportverzögerung. Reale Gatter können sehr kurze Impulse dämpfen. Die 3 ns sind deshalb ein Ergebnis dieses Modells, keine universelle Hardwarekonstante.

Buchbezug: Eigene Python-Ergänzung zu DDCA 2.9.2. Abbildung: ausgeführtes Lehrmodell. Eigene Erklärungen und Beispiele.

06 / VORHER UND NACHHER IN NGSPICE

Ein reproduzierbarer Versuch

hazard_rc.cir enthält beide Varianten gleichzeitig. $B=C=1$; A fällt nach 20 ns. Verhaltensquellen bilden die Logik ab, Widerstände und Kondensatoren erzeugen endliche Flanken. Pegel: 0 und 1 V; Entscheidungsschwelle: 0,5 V. **Das ist ein RC-Lehrmodell, keine CMOS-Transistorschaltung.** Es dient dazu, ungleiche Laufzeiten sichtbar zu machen. Die RC-Zeitkonstante ist nicht automatisch die Verzögerung am 50%-Punkt.



Was die Messungen bedeuten

glitch_start und glitch_end sind die beiden 0,5-V-Durchgänge von Y. glitch_width ist deren zeitlicher Abstand. Der vorbereitende Testlauf ergab etwa 2,78 ns. fixed_min prüft die niedrigste Spannung des korrigierten Ausgangs zwischen 15 und 40 ns; hier blieb er bei 1 V.

Warum nicht exakt 3 ns wie in Python?

Python verwendet feste ganzzahlige Laufzeiten und ideale Sprünge. SPICE berechnet kontinuierliche RC-Flanken und schaltet an einer Spannungsschwelle. Die Modelle zeigen denselben Mechanismus, müssen aber nicht dieselbe Impulsbreite liefern.

Die bereitgestellten Wellenformen sind Referenzläufe zur Vorbereitung. In deiner Auswertung dokumentierst du deinen eigenen Lauf, deine Beobachtungen und Änderungen.

Buchbezug: Eigener ngspice-Versuch zu DDCA 2.9.2; Kapitel 2.9 insgesamt S. 86–93 (PDF 110–117). Eigene Erklärungen und Beispiele.